Graph Data Structure Study Guide

1. Introduction to Graph

A **Graph** is a non-linear data structure consisting of nodes (vertices) and edges that connect these nodes. Unlike trees, graphs can have cycles and multiple paths between nodes. Graphs are used to represent relationships between objects, making them fundamental in computer science for solving complex problems.

Key characteristics of graphs:

- Contains vertices (nodes) and edges (connections)
- Can be finite or infinite
- Edges can be directed or undirected
- May contain cycles
- Can be weighted or unweighted

2. Types of Graph

2.1 Based on Direction

Undirected Graph

- Edges have no direction
- If there's an edge from A to B, there's also an edge from B to A
- Represented as (A, B) where order doesn't matter

Directed Graph (Digraph)

- Edges have direction
- Edge from A to B doesn't necessarily mean edge from B to A
- Represented as (A → B)

2.2 Based on Weights

Weighted Graph

- Each edge has an associated weight or cost
- Used in shortest path algorithms

Unweighted Graph

- All edges are considered equal
- No weights assigned to edges

2.3 Based on Connectivity

Connected Graph

- There exists a path between every pair of vertices

Disconnected Graph

- Some vertices are not reachable from others

2.4 Special Types

Complete Graph

- Every vertex is connected to every other vertex
- Has maximum possible edges: n(n-1)/2 for undirected graphs

Cyclic Graph

- Contains at least one cycle

Acyclic Graph

- Contains no cycles
- DAG (Directed Acyclic Graph) is commonly used

3. Application and Usage

Real-world Applications

Social Networks

- Facebook friend connections
- Twitter follower relationships
- LinkedIn professional networks

Transportation Systems

- Road networks and GPS navigation
- Flight routes and connections
- Public transportation systems

Computer Networks

- Internet topology
- Local area networks
- Router connections

Web and Internet

- Web page linking structure
- Search engine algorithms (PageRank)
- Website navigation

Game Development

- Pathfinding algorithms
- Game state representations
- AI decision trees

Database Systems

- Relationship modeling
- Query optimization
- Dependency tracking

4. Terminology related to Graph

Vertex (Node)

- Basic unit of a graph
- Represents an entity or object

Edge (Arc)

- Connection between two vertices
- Represents relationship between entities

Adjacent Vertices

- Two vertices connected by an edge
- Also called neighbors

Degree of Vertex

- Number of edges connected to a vertex
- In-degree: incoming edges (directed graphs)
- Out-degree: outgoing edges (directed graphs)

Path

- Sequence of vertices connected by edges
- Simple path: no repeated vertices

Cycle

- Path that starts and ends at the same vertex
- Simple cycle: no repeated vertices except start/end

Connected Components

- Maximal set of vertices where each pair is connected by a path

Spanning Tree

- Subgraph that includes all vertices with minimum edges to keep graph connected

Graph Traversal

- Process of visiting all vertices in a systematic manner
- Common methods: BFS (Breadth-First Search) and DFS (Depth-First Search)

5. Representation and Construction of Graph

5.1 Adjacency Matrix Representation

A 2D array where matrix[i][j] = 1 if there's an edge from vertex i to vertex j.

```java
public class GraphAdjacencyMatrix {
    private int[][] matrix;
    private int vertices;

    public GraphAdjacencyMatrix(int vertices) {
        this.vertices = vertices;
        this.matrix = new int[vertices][vertices];
    }

    // Add edge between two vertices
    public void addEdge(int source, int destination) {
        matrix[source][destination] = 1;
        matrix[destination][source] = 1; // For undirected graph
    }

    // Remove edge between two vertices
    public void removeEdge(int source, int destination) {
        matrix[source][destination] = 0;
        matrix[destination][source] = 0; // For undirected graph
    }

    // Check if edge exists
    public boolean hasEdge(int source, int destination) {
        return matrix[source][destination] == 1;
    }

    // Display the matrix
    public void displayMatrix() {
        for (int i = 0; i < vertices; i++) {
            for (int j = 0; j < vertices; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Advantages:

- Easy to implement
- Quick edge lookup O(1)
- Good for dense graphs

Disadvantages:

- Space complexity $O(V^2)$
- Inefficient for sparse graphs

5.2 Adjacency List Representation

Array of lists where each list contains neighbors of a vertex.

java

```java
import java.util.*;

public class GraphAdjacencyList {
    private int vertices;
    private ArrayList<ArrayList<Integer>> adjacencyList;

    public GraphAdjacencyList(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>();

        // Initialize adjacency list
        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    // Add edge between two vertices
    public void addEdge(int source, int destination) {
        adjacencyList.get(source).add(destination);
        adjacencyList.get(destination).add(source); // For undirected graph
    }

    // Remove edge between two vertices
    public void removeEdge(int source, int destination) {
        adjacencyList.get(source).remove(Integer.valueOf(destination));
        adjacencyList.get(destination).remove(Integer.valueOf(source));
    }

    // Get neighbors of a vertex
    public ArrayList<Integer> getNeighbors(int vertex) {
        return adjacencyList.get(vertex);
    }

    // Display the graph
    public void displayGraph() {
        for (int i = 0; i < vertices; i++) {
            System.out.print("Vertex " + i + ": ");
            for (int neighbor : adjacencyList.get(i)) {
                System.out.print(neighbor + " ");
            }
            System.out.println();
        }
    }

    // Depth-First Search traversal
    public void DFS(int startVertex) {
```

```java
        boolean[] visited = new boolean[vertices];
        DFSUtil(startVertex, visited);
    }

    private void DFSUtil(int vertex, boolean[] visited) {
        visited[vertex] = true;
        System.out.print(vertex + " ");

        for (int neighbor : adjacencyList.get(vertex)) {
            if (!visited[neighbor]) {
                DFSUtil(neighbor, visited);
            }
        }
    }

    // Breadth-First Search traversal
    public void BFS(int startVertex) {
        boolean[] visited = new boolean[vertices];
        Queue<Integer> queue = new LinkedList<>();

        visited[startVertex] = true;
        queue.offer(startVertex);

        while (!queue.isEmpty()) {
            int vertex = queue.poll();
            System.out.print(vertex + " ");

            for (int neighbor : adjacencyList.get(vertex)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.offer(neighbor);
                }
            }
        }
    }
}
```

Advantages:

- Space efficient for sparse graphs O(V + E)
- Easy to iterate through neighbors
- Dynamic size

Disadvantages:

- Edge lookup takes O(V) time

- More complex implementation

5.3 Edge List Representation

Simple list of all edges in the graph.

```java
import java.util.*;

class Edge {
    int source, destination, weight;

    public Edge(int source, int destination, int weight) {
        this.source = source;
        this.destination = destination;
        this.weight = weight;
    }
}

public class GraphEdgeList {
    private List<Edge> edges;
    private int vertices;

    public GraphEdgeList(int vertices) {
        this.vertices = vertices;
        this.edges = new ArrayList<>();
    }

    public void addEdge(int source, int destination, int weight) {
        edges.add(new Edge(source, destination, weight));
    }

    public void displayEdges() {
        for (Edge edge : edges) {
            System.out.println(edge.source + " -> " + edge.destination +
                               " (weight: " + edge.weight + ")");
        }
    }
}
```

5.4 Complete Graph Implementation Example

```java
public class GraphExample {
    public static void main(String[] args) {
        // Create a graph with 5 vertices
        GraphAdjacencyList graph = new GraphAdjacencyList(5);

        // Add edges
        graph.addEdge(0, 1);
        graph.addEdge(0, 2);
        graph.addEdge(1, 3);
        graph.addEdge(2, 4);
        graph.addEdge(3, 4);

        // Display graph
        System.out.println("Graph representation:");
        graph.displayGraph();

        // Perform traversals
        System.out.println("\nDFS traversal starting from vertex 0:");
        graph.DFS(0);

        System.out.println("\n\nBFS traversal starting from vertex 0:");
        graph.BFS(0);
    }
}
```

Quiz Questions

Question 1: What is the maximum number of edges in a simple undirected graph with n vertices? a) n b) n(n-1) c) n(n-1)/2 d) n^2

Correct Answer: c) n(n-1)/2

Question 2: In which graph representation is checking if an edge exists between two vertices most efficient? a) Adjacency List b) Edge List c) Adjacency Matrix d) All are equally efficient

Correct Answer: c) Adjacency Matrix

Question 3: What is the space complexity of an adjacency list representation for a graph with V vertices and E edges? a) O(V) b) O(E) c) O(V + E) d) $O(V^2)$

Correct Answer: c) O(V + E)

Question 4: In a directed graph, if vertex A has an in-degree of 3 and out-degree of 2, how many edges are connected to vertex A? a) 5 b) 3 c) 2 d) 1

Correct Answer: a) 5

Question 5: Which traversal algorithm uses a queue data structure? a) Depth-First Search (DFS) b) Breadth-First Search (BFS) c) Both DFS and BFS d) Neither DFS nor BFS

Correct Answer: b) Breadth-First Search (BFS)

Practice Coding Questions

Question 1: Implement a method `findPath(int start, int end)` in the GraphAdjacencyList class that returns true if there exists a path between two given vertices using DFS. The method should return false if no path exists.

Question 2: Write a Java program to detect if a given undirected graph contains a cycle. Create a class `CycleDetection` with a method `hasCycle()` that returns true if the graph has a cycle, false otherwise.

Question 3: Implement a method `shortestPath(int start, int end)` that finds the shortest path between two vertices in an unweighted graph using BFS. The method should return the minimum number of edges in the path, or -1 if no path exists.

Question 4: Create a method `getConnectedComponents()` that returns the number of connected components in an undirected graph. Also, implement a method `printConnectedComponents()` that prints all vertices in each connected component.

Question 5: Write a Java program to implement a weighted graph using adjacency list representation. Include methods to add weighted edges, find the vertex with minimum weight from a given vertex, and display the graph with weights.